OBJECT ORIENTED PROGRAMMING

Programming for Data Science

Classes and Objects

- Python is an object oriented programming language.
- Almost everything in Python is an object, with its properties and methods.
- A Class is like an object constructor, or a "blueprint" for creating objects.

Classes and Objects

```
class Person:
  x = 5

  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("John", 36)

print(p1.x)
print(p1.name)
print(p1.age)
```

Classes and Objects

To create a class, use the keyword ***class***

```
class Person:
  x = 5

  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("John", 36)

print(p1.x)
print(p1.name)
print(p1.age)
```

Classes and Objects

To create a class, use the keyword ***class***

Create a class named Person, with a property named ***x,*** use the ***__init__()*** function to assign values for name and age

```
class Person:
  x = 5

  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("John", 36)

print(p1.x)
print(p1.name)
print(p1.age)
```

Classes and Objects

To create a class, use the keyword ***class***

```
class Person:
  x = 5

  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("John", 36)

print(p1.x)
print(p1.name)
print(p1.age)
```

Create a class named Person, with a property named ***x,*** use the ***__init__()*** function to assign values for name and age

The ***__init__()*** function is called automatically every time the class is being used to create a new object.

Classes and Objects

```
class Person:
  x = 5

  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("John", 36)

print(p1.x)
print(p1.name)
print(p1.age)
```

To create a class, use the keyword ***class***

Create a class named Person, with a property named ***x,*** use the ***__init__()*** function to assign values for name and age

The ***__init__()*** function is called automatically every time the class is being used to create a new object.

Create an object named **p1**, and print the values of ***x, name,*** and ***age***

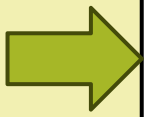
Classes and Objects

```
class Person:
  x = 5

  def __init__(self, name, age):
    self.name = name
    self.age = age

p1 = Person("John", 36)

print(p1.x)
print(p1.name)
print(p1.age)
```



Output

```
5
John
36
```

Class

__str__() Function

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def __str__(self):
    return f"{self.name}
              ({self.age})"

p1 = Person("John", 36)

print(p1)
```

Output

```
John(36)
```

The ***__str__()*** function controls what should be returned when the class object is represented as a string.

__str__() Function

The ***__str__()*** function controls what should be returned when the class object is represented as a string.

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def __str__(self):
    return f"{self.name}
              ({self.age})"

p1 = Person("John", 36)

print(p1)
```

Output

```
John(36)
```

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age


p1 = Person("John", 36)


print(p1)
```

Output

```
<__main__.Person object at
0x15039e602100>
```

Object Methods

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def myfunc1(abc):
    print("Hello my name is " + abc.name)


  def myfunc2(self, name):
    self.name = name

p1 = Person("John", 36)
p1.myfunc1()
p1.myfunc2("John Cena")
print("My new name is " + p1.name)
```

Object Methods

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def myfunc1(abc):
    print("Hello my name is " + abc.name)


  def myfunc2(self, name):
    self.name = name

p1 = Person("John", 36)
p1.myfunc1()
p1.myfunc2("John Cena")
print("My new name is " + p1.name)
```

The ***self*** parameter is a reference to the current instance of the class, and is used to access variables that belong to the class.

It does not have to be named ***self***, you can call it whatever you like, but it has to be the first parameter of any function in the class.

Object Methods

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def myfunc1(abc):
    print("Hello my name is " + abc.name)


  def myfunc2(self, name):
    self.name = name

p1 = Person("John", 36)
p1.myfunc1()
p1.myfunc2("John Cena")
print("My new name is " + p1.name)
```

The ***self*** parameter is a reference to the current instance of the class, and is used to access variables that belong to the class.

It does not have to be named ***self***, you can call it whatever you like, but it has to be the first parameter of any function in the class.

Output

```
Hello my name is John
My new name is John Cena
```

Modify & Delete Object Properties

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def myfunc(self):
    print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.name = "Alex"
del p1.age

print(p1.name)
print(p1.age)
```

Modify & Delete Object Properties

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def myfunc(self):
    print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.name = "Alex"
del p1.age

print(p1.name)
print(p1.age)
```

Output

```
Alex
Traceback (most recent call last):
  File "./prog.py", line 14, in
<module>
AttributeError: 'Person' object has
no attribute 'age'
```

Modify & Delete Object Properties

```
class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def myfunc(self):
    print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.name = "Alex"
del p1.age

print(p1.name)
print(p1.age)
```

You can delete objects by using the ***del*** keyword

```
del p1
```

Output

```
Alex
Traceback (most recent call last):
  File "./prog.py", line 14, in
<module>
AttributeError: 'Person' object has
no attribute 'age'
```

pass Statement

Class definitions cannot be empty, but if you for some reason have a class definition with no content, put in the **pass** statement to avoid getting an error.

```
class Person:
  pass
```

Inheritance

- **Inheritance** allows us to define a class that inherits all the methods and properties from another class.
- **Parent class** is the class being inherited from, also called base class.
- **Child class** is the class that inherits from another class, also called derived class.

Inheritance

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  pass

x = Person("John", "Doe")
x.printname()
x = Student("Mike", "Olsen")
x.printname()
```

Output

```
John Doe
Mike Olsen
```

Inheritance

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)

x = Student("Mike", "Olsen")
x.printname()
```

Inheritance

When you add the ***__init__()*** function, the child class will no longer inherit the parent's ***__init__()*** function.

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)

x = Student("Mike", "Olsen")
x.printname()
```

Inheritance

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)

x = Student("Mike", "Olsen")
x.printname()
```

When you add the ***__init__()*** function, the child class will no longer inherit the parent's ***__init__()*** function.

The child's ***__init__()*** function overrides the inheritance of the parent's ***__init__()*** function.

Inheritance

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)

x = Student("Mike", "Olsen")
x.printname()
```

When you add the ***__init__()*** function, the child class will no longer inherit the parent's ***__init__()*** function.

The child's ***__init__()*** function overrides the inheritance of the parent's ***__init__()*** function.

To keep the inheritance of the parent's ***__init__()*** function, add a call to the parent's ***__init__()*** function.

super().__init__(fname, lname)
Person.__init__(self, fname, lname)

Inheritance

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname):
    super().__init__(fname, lname)

x = Student("Mike", "Olsen")
x.printname()
```

Output

```
Mike Olsen
```

Add Properties and Methods

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname, year):
    super().__init__(fname, lname)
    self.graduationyear = year

  def welcome(self):
    print("Welcome", self.firstname, self.lastname, "to the
class of", self.graduationyear)

x = Student("Mike", "Olsen", 2024)
x.welcome()
```

Add Properties and Methods

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)

class Student(Person):
  def __init__(self, fname, lname, year):
    super().__init__(fname, lname)
    self.graduationyear = year

  def welcome(self):
    print("Welcome", self.firstname, self.lastname, "to the
class of", self.graduationyear)

x = Student("Mike", "Olsen", 2024)
x.welcome()
```

Output

```
Welcome Mike Olsen to
the class of 2024
```

Polymorphism

- The word "polymorphism" means "many forms", and in programming it refers to methods/functions/operators with the same name that can be executed on many objects or classes.

Function Polymorphism

- An example of a Python function that can be used on different objects is the ***len()*** function.

```
x = "Hello World!"
mytuple = ("apple", "banana", "cherry", "kiwi")
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}

print(len(x))
print(len(mytuple))
print(len(thisdict))
```

Output

```
12
4
3
```

Class Polymorphism

- Polymorphism is often used in Class methods, where we can have multiple classes with the same method name.

Class Polymorphism

```
class Car:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Drive!")

class Boat:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Sail!")

class Plane:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Fly!")
```

Class Polymorphism

```
#Create a Car object
car1 = Car("Ford", "Mustang")

#Create a Boat object
boat1 = Boat("Ibiza", "Touring 20")

#Create a Plane object
plane1 = Plane("Boeing", "747")

for x in (car1, boat1, plane1):
  x.move()
```



```
class Car:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Drive!")

class Boat:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Sail!")

class Plane:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Fly!")
```

Class Polymorphism

```
#Create a Car object
car1 = Car("Ford", "Mustang")

#Create a Boat object
boat1 = Boat("Ibiza", "Touring 20")

#Create a Plane object
plane1 = Plane("Boeing", "747")

for x in (car1, boat1, plane1):
  x.move()
```

Output

```
Drive!
Sail!
Fly!
```

```
class Car:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Drive!")

class Boat:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Sail!")

class Plane:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Fly!")
```

Inheritance Class Polymorphism

```
class Vehicle:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Move!")

class Car(Vehicle):
  pass

class Boat(Vehicle):
  def move(self):
    print("Sail!")

class Plane(Vehicle):
  def move(self):
    print("Fly!")
```

Inheritance Class Polymorphism

```
class Vehicle:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Move!")

class Car(Vehicle):
  pass

class Boat(Vehicle):
  def move(self):
    print("Sail!")

class Plane(Vehicle):
  def move(self):
    print("Fly!")
```



```
#Create a Car object
car1 = Car("Ford", "Mustang")

#Create a Boat object
boat1 = Boat("Ibiza", "Touring 20")

#Create a Plane object
plane1 = Plane("Boeing", "747")

for x in (car1, boat1, plane1):
  print(x.brand, x.model, end=" ")
  x.move()
  print()
```

Inheritance Class Polymorphism

```
class Vehicle:
  def __init__(self, brand, model):
    self.brand = brand
    self.model = model

  def move(self):
    print("Move!")

class Car(Vehicle):
  pass

class Boat(Vehicle):
  def move(self):
    print("Sail!")

class Plane(Vehicle):
  def move(self):
    print("Fly!")
```

```
#Create a Car object
car1 = Car("Ford", "Mustang")

#Create a Boat object
boat1 = Boat("Ibiza", "Touring 20")

#Create a Plane object
plane1 = Plane("Boeing", "747")

for x in (car1, boat1, plane1):
  print(x.brand, x.model, end=" ")
  x.move()
  print()
```

Output

```
Ford Mustang Move!

Ibiza Touring 20 Sail!

Boeing 747 Fly!
```

Thank you!